

Technische Dokumentation

Architektur, Datenbankschema, API und Deployment

PROJEKT

Lernzeit-Manager
ISEF01 · Projekt Software Engineering

STAND

21. August 2026

TEAM

Elias Ebertshäuser • Assis Ramadan • Julian Wagner

1. Systemarchitektur

1.1 Übersicht

Der Lernzeit-Manager folgt einer klassischen **dreischichtigen Client-Server-Architektur**:

```
[Browser] ↔ [Flask-Backend (REST-API)] ↔ [PostgreSQL-Datenbank]
Angular SPA   Python 3.12, Port 5000   Relationale Datenbank
```

- **Frontend (Angular 22 SPA):** Eine Single-Page-Aplication, die vollständig im Browser läuft. Sie kommuniziert ausschließlich über HTTP-REST mit dem Backend.
- **Backend (Flask 3.1 REST-API):** Eine Python-Anwendung, die Authentifizierung, Datenlogik und Datenbankzugriff übernimmt. Sie gibt JSON zurück und liefert in Produktion zusätzlich die gebauten Angular-Dateien aus (siehe Abschnitt 5.2).
- **Datenbank (PostgreSQL 16):** Speichert alle Nutzer-, Ziel-, Planungs-, Zwischenziel- und Session-Daten dauerhaft.

1.2 Komponentendiagramm (Textbeschreibung)



■ 1.3 Kommunikation

Das Frontend sendet alle API-Anfragen an relative URLs (z. B. `/api/goals`). Im Entwicklungsmodus leitet ein Angular-Dev-Proxy diese an `http://localhost:5000` weiter. In Produktion liefert Flask sowohl die API als auch die statischen Angular-Dateien aus demselben Server aus — es gibt keine Cross-Origin-Problematik.

Alle Anfragen an geschützte Endpunkte enthalten einen JWT-Token im HTTP-Header:

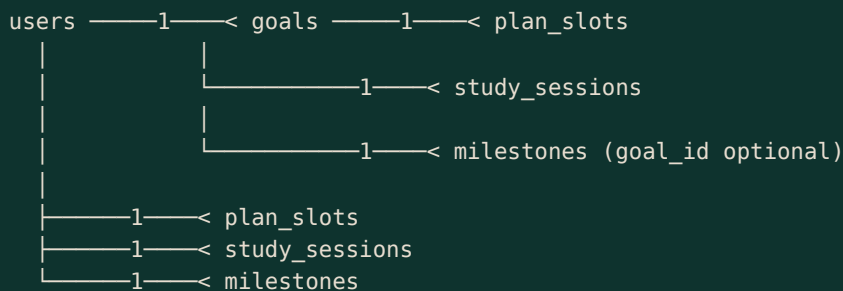
```
Authorization: Bearer <token>
```

Der Token wird beim Login/bei der Registrierung ausgestellt, im `localStorage` des Browsers unter dem Schlüssel `lm_token` gespeichert und ist 8 Stunden gültig (`JWT_ACCESS_TOKEN_EXPIRES`, `backend/app/config.py`). Ein Angular-Interceptor (`auth.interceptor.ts`) hängt ihn automatisch an jede ausgehende Anfrage an. Ein Seiten-Reload übersteht die Anmeldung, weil die Anwendung den gespeicherten Token beim Start über `GET /api/auth/me` prüft und nur bei einer ausdrücklichen HTTP-401/403-Ablehnung abmeldet.

Alle vom Server ausgelieferten Zeitstempel sind UTC und tragen ein angehängtes `Z` (`backend/app/time_utils.py`, Funktion `iso_utc`), damit der Browser sie nicht fälschlich als Ortszeit interpretiert.

2. Datenbankschema

2.1 Entity-Relationship-Übersicht



Jeder Nutzer besitzt beliebig viele Ziele, Planungseinträge, Lernsessions und Zwischenziele. Ein Zwischenziel kann, muss aber nicht einem Lernziel zugeordnet sein (`goal_id` ist nullable). Beim Löschen eines Nutzers werden alle zugehörigen Datensätze automatisch mitgelöscht (Cascade Delete, `cascade="all, delete-orphan"` auf allen vier Relationships in `backend/app/models/user.py`). Beim Löschen eines Ziels gilt das nur für dessen Planungseinträge und Lernsessions (`cascade="all, delete-orphan"` in `backend/app/models/goal.py`); die Relationship `Goal.milestones` trägt bewusst **kein** Cascade. Zugehörige Zwischenziele bleiben deshalb bestehen und verlieren nur ihre Zuordnung (`goal_id` wird `null`) — belegt durch `backend/tests/test_milestones.py::test_deleting_a_goal_keeps_its_milestones` (siehe Geschäftsregel GR-2 in der Fachlichen Dokumentation).

Das Schema entsteht aus vier Migrationen (`backend/migrations/versions/`), die in dieser Reihenfolge angewendet werden:

MIGRATION	INHALT
<code>0001_ms4_initial_schema.py</code>	Legt <code>users</code> , <code>goals</code> , <code>plan_slots</code> , <code>study_sessions</code> mit den Basisspalten an
<code>0002_goal_prioritaet_ergebnis.py</code>	Ergänzt <code>goals</code> um <code>priority</code> , <code>grade</code> , <code>result_note</code> (alle nullable)
<code>0003_milestones.py</code>	Legt die Tabelle <code>milestones</code> an
<code>0004_goal_workload_hours.py</code>	Ergänzt <code>goals</code> um <code>workload_hours</code> (nullable)

2.2 Tabellenbeschreibung

users

SPALTE	TYP	BESCHREIBUNG
<code>id</code>	INTEGER PK	Primärschlüssel
<code>email</code>	VARCHAR(255) UNIQUE	E-Mail-Adresse (Login)

SPALTE	TYP	BESCHREIBUNG
name	VARCHAR(255)	Anzeigenname
password_hash	VARCHAR(255)	Werkzeug-Hash des Passworts

goals

SPALTE	TYP	BESCHREIBUNG
id	INTEGER PK	Primärschlüssel
user_id	INTEGER FK→users	Besitzer
title	VARCHAR(255)	Titel des Lernziels
module_name	VARCHAR(255)	Modulkürzel oder -name
ects	INTEGER	ECTS-Punkte (1–30), Standard 5
workload_hours	INTEGER NULL	Manueller Lernaufwand in Stunden (1–1000); überschreibt, falls gesetzt, <code>ects * 30</code>
status	VARCHAR(50)	open / in_progress / achieved
priority	VARCHAR(10) NULL	high / medium / low
grade	VARCHAR(10) NULL	Note, frei eintragbar
result_note	VARCHAR(500) NULL	Ergebnis-Notiz
target_date	DATE	Angestrebter Abschlusstermin
created_at	DATETIME	Anlage-Zeitpunkt

plan_slots

SPALTE	TYP	BESCHREIBUNG
id	INTEGER PK	Primärschlüssel
user_id	INTEGER FK→users	Besitzer
goal_id	INTEGER FK→goals	Zugehöriges Lernziel
year	INTEGER	Jahr der Planung
month	INTEGER	Monat der Planung (1–12)
day	INTEGER NULL	Optionaler Tag (1–31)
planned_time	VARCHAR(5) NULL	Optionale Uhrzeit (HH:MM)

SPALTE	TYP	BESCHREIBUNG
duration_minutes	INTEGER	Geplante Lernzeit in Minuten (5–480)
note	VARCHAR(500) NULL	Optionale Notiz

study_sessions

SPALTE	TYP	BESCHREIBUNG
id	INTEGER PK	Primärschlüssel
user_id	INTEGER FK→users	Besitzer
goal_id	INTEGER FK→goals	Zugehöriges Lernziel
started_at	DATETIME	UTC-Startzeit (naiv)
paused_at	DATETIME NULL	Zeitpunkt der aktuellen Pause
total_paused_seconds	INTEGER	Summierte Pausendauer aller Pausen
ended_at	DATETIME NULL	UTC-Endzeit
duration_seconds	INTEGER NULL	Berechnete Netto-Lernzeit
status	VARCHAR(20)	active / paused / completed
note	TEXT NULL	Optionale Notiz beim Stoppen

milestones

SPALTE	TYP	BESCHREIBUNG
id	INTEGER PK	Primärschlüssel
user_id	INTEGER FK→users	Besitzer
goal_id	INTEGER FK→goals, NULL	Optional zugeordnetes Lernziel
title	VARCHAR(200)	Titel des Zwischenziels
year	INTEGER	Zugehöriges Jahr
month	INTEGER	Zugehöriger Monat (1–12)
due_day	INTEGER NULL	Optionaler Fälligkeitstag im Monat
done	BOOLEAN	Erledigt-Status, Standard <code>false</code>
created_at	DATETIME	Anlage-Zeitpunkt

3. API-Beschreibung

Alle Endpunkte liegen unter dem Präfix `/api`. Geschützte Endpunkte erfordern den `Authorization: Bearer <token>`-Header und liefern bei fehlendem/ungültigem Token HTTP 401. Eingabefehler liefern durchgängig HTTP 400 mit `{"error": "..."} (zentral behandelt über ValidationError in backend/app/validation.py, übersetzt in backend/app/__init__.py).` Zugriffe auf fremde oder nicht existierende Ressourcen liefern HTTP 404, nie 403 (GR-6).

■ 3.1 Authentifizierung

POST /api/auth/register — Neuen Nutzer registrieren. Kein Token erforderlich.

Request: `{ "email": "...", "name": "...", "password": "..." }`

Response (201): `{ "access_token": "...", "user": { "id": 1, "email": "...", "name": "..." } }`

Fehler: 400 (Validierung, siehe README-Abschnitt „Geltende Wertebereiche der API“), 409 (E-Mail bereits registriert)

POST /api/auth/login — Mit Zugangsdaten einloggen. Kein Token erforderlich.

Request: `{ "email": "...", "password": "..." }`

Response (200): `{ "access_token": "...", "user": { ... } }`

Fehler: 400, 401 (Falsches Passwort oder unbekannte E-Mail)

GET /api/auth/me — Daten des eingeloggten Nutzers abrufen. Token erforderlich.

Response (200): `{ "id": 1, "email": "...", "name": "..." }`

Fehler: 401

■ 3.2 Lernziele

GET /api/goals — Alle Ziele des Nutzers, sortiert nach Zieldatum.

POST /api/goals — Neues Ziel anlegen.

Request: `{ "title", "module_name", "target_date", "ects" (optional, Standard 5), "workload_hours" (optional, 1–1000), "priority" (optional), "grade" (optional), "result_note" (optional) }`

Response (201): das erstellte Goal-Objekt, alle Spalten aus 2.2 als JSON.

PUT /api/goals/<id> — Ziel bearbeiten. Nur die im Body enthaltenen Felder werden geändert (`title`, `module_name`, `target_date`, `ects`, `workload_hours`, `status`, `priority`, `grade`, `result_note`).

DELETE /api/goals/<id> — Ziel löschen (inkl. Sessions und Planungseinträgen, Cascade Delete; zugehörige Zwischenziele bleiben bestehen und verlieren nur ihre Zuordnung, siehe Abschnitt 2.1). Response (204).

■ 3.3 Planung

GET /api/plans — Planungseinträge abrufen. Query-Parameter (alle optional): `goal_id`, `year`, `month`.

GET /api/plans/proposal — Grobplanungs-Vorschlag (FR-2.1, FR-2.2, FR-3.3). Query-Parameter `year` / `month` (beide zusammen oder keiner, sonst gilt der laufende Monat). Response (200) je Ziel:

`weekly_budget_minutes`, `suggested_month_minutes`, `planned_minutes`, `deviation_minutes` (siehe Fachliche Dokumentation, Abschnitt 3.4).

POST /api/plans — Neuen Planungseintrag anlegen. Request: `{ "goal_id", "year", "month", "day" (optional), "planned_time" (optional), "duration_minutes" (Standard 60), "note" (optional) }`.

POST /api/plans/series — Serientermine anlegen (FR-3.1, Plan P9). Request wie `POST /api/plans`, zusätzlich `"days"`: eine Liste mit 1–31 eindeutigen Tagen. Legt in einer Transaktion für jeden Tag einen PlanSlot mit denselben Uhrzeit-/Dauer-/Notiz-Werten an. Response (201): Liste der erstellten PlanSlots.

PUT /api/plans/<id> — Planungseintrag bearbeiten (`day`, `planned_time`, `duration_minutes`, `note`).

DELETE /api/plans/<id> — Planungseintrag löschen. Response (204).

■ 3.4 Zwischenziele (Plan P5, FR-3.2)

GET /api/milestones — Zwischenziele abrufen. Query-Parameter (optional): `goal_id`, `year`, `month`.

POST /api/milestones — Neues Zwischenziel anlegen. Request: `{ "title", "year", "month", "due_day" (optional), "goal_id" (optional), "done" (optional, Standard false) }`.

PUT /api/milestones/<id> — Zwischenziel bearbeiten (`title`, `due_day`, `done`).

DELETE /api/milestones/<id> — Zwischenziel löschen. Response (204).

■ 3.5 Lernsessions

GET /api/sessions/active — Aktive oder pausierte Session abrufen. Response (200): Session-Objekt mit zusätzlichem `goal_title`. Response (204): keine aktive Session.

GET /api/sessions — Abgeschlossene und laufende Sessions abrufen. Query-Parameter (optional): `goal_id`, `limit` (Standard 50, maximal 200).

POST /api/sessions/start — Neue Session starten. Request: `{ "goal_id" }`. Fehler: 409, wenn bereits eine Session aktiv/pausiert ist (liefert die existierende Session mit).

POST /api/sessions/<id>/pause — Session pausieren. Fehler: 409 (nicht aktiv).

POST /api/sessions/<id>/resume — Pausierte Session fortsetzen; Response enthält die aktualisierten `total_paused_seconds`. Fehler: 409 (nicht pausiert).

POST /api/sessions/<id>/stop — Session beenden, Netto-Dauer berechnen. Request (optional): `{ "note" }`. Fehler: 409 (bereits beendet).

■ 3.6 Dashboard

GET /api/dashboard — Aggregierte Daten für die Startseite. Response (200, gekürzt):

```
{
  "current_month": { "year": 2026, "month": 8, "planned_minutes": 300,
    "actual_minutes": 120, "paused_minutes": 15 },
  "goals": [ { ...Goal-Felder..., "total_actual_minutes": 120,
    "planned_ects_minutes": 9000, "weekly_budget_minutes": 900,
    "milestones": [ ...Milestone-Objekte des laufenden Monats... ] } ],
  "weekly_history": [ { "week_start": "2026-07-06", "minutes": 240 }, ... ],
```



```

"deadline_warnings": [ { "goal_id": 1, "title": "...", "days_left": 5,
  "progress_pct": 30 } ],
"milestones": { "done": 1, "total": 4 },
"inactivity_warning": false,
"reminder_text": null,
"active_session": null
}

```

`planned_ects_minutes` und `weekly_budget_minutes` berücksichtigen einen gesetzten `workload_hours` - Override (siehe `Goal.effective_workload_minutes()` in `backend/app/models/goal.py`). `weekly_history` deckt die letzten 8 Kalenderwochen ab (FR-6.3).

■ 3.7 Auswertung (Plan P11, FR-6.4, FR-5.3)

GET /api/stats — Response (200, gekürzt):

```

{
  "per_goal": [ { "goal_id", "title", "module_name", "planned_ects_minutes",
    "total_actual_minutes", "progress_pct", "ampel": "gruen"|"gelb"|"rot" } ],
  "per_month": [ { "year", "month", "planned_minutes", "actual_minutes" } ],
  "by_daytime": { "morning_minutes", "afternoon_minutes", "evening_minutes",
    "night_minutes" },
  "achieved_goals": [ { "goal_id", "title", "module_name", "grade",
    "result_note", "target_date" } ]
}

```

`per_month` deckt die letzten 6 Kalendermonate einschließlich des laufenden ab. Die Ampel je Modul folgt denselben Schwellen wie der Fortschrittsbalken im Dashboard (≥ 100 % grün, ≥ 50 % gelb, sonst rot).

■ 3.8 Health Check

GET /api/health — Response (200): `{ "status": "ok" }`. Kein Token erforderlich.

4. Frontend-Architektur

4.1 Verzeichnisstruktur

```
frontend/src/app/
├── core/
│   ├── models/index.ts      TypeScript-Interfaces (Goal, PlanSlot, Milestone, ...)
│   ├── services/            HTTP-Services: Auth-, Goal-, Plan-, Milestone-,
│   │                       Session-, Dashboard-, Stats-, Reminder-Service
│   ├── interceptors/       auth.interceptor.ts (JWT-Header-Injektion)
│   ├── guards/             auth.guard.ts (Routen-Schutz)
│   ├── validation.ts        Client-seitige Spiegelung der Server-Validierung
│   ├── upcoming-slot.ts     Reine Funktion für die FR-7.2-Erinnerung
│   ├── goal-delete-confirm.ts Text für die Löschen-Sicherheitsabfrage
│   └── token-storage.ts     Kapselt den Zugriff auf localStorage (`lm_token`)
├── features/
│   ├── auth/               Login- und Registrierungskomponenten
│   ├── dashboard/          Dashboard-Komponente (Zwei-Spalten-Layout ab ~1000px)
│   ├── goals/              Lernziel-Verwaltung (Anlegen + Bearbeiten)
│   ├── planning/           Planung, Grobplanung, Serientermine (day-picker.ts)
│   ├── timer/              Timer-Komponente
│   ├── calendar/           Kalender-Tab (Monatsraster)
│   └── stats/              Auswertungs-Tab
├── shared/
│   └── week-chart.ts        Geteiltes Wochendiagramm (Dashboard + Auswertung)
├── layout/
└── navbar/                 Navigationsleiste inkl. Erinnerungs-Glocke/Dropdown
```

4.2 Wichtige Konzepte

Standalone-Komponenten: Angular 22 verwendet keine NgModule mehr. Jede Komponente deklariert ihre Abhängigkeiten (`imports: [RouterLink, FormsModule, ...]`) direkt.

Signals: Reaktive Zustände werden über `signal<T>()` verwaltet. Ändert sich ein Signal, aktualisiert Angular nur die betroffene Stelle im DOM.

Lazy Loading: Alle Feature-Routen (`frontend/src/app/app.routes.ts`) werden erst bei Bedarf per `loadComponent` geladen, was die initiale Ladezeit der App verkürzt.

JWT-Interceptor: Ein funktionaler Angular-Interceptor (`HttpInterceptorFn`) hängt automatisch den JWT-Token an jede ausgehende HTTP-Anfrage an.

Auth-Guard: Eine `CanActivateFn`-Funktion prüft bei jedem Seitenwechsel, ob ein Token im `localStorage` vorhanden ist. Fehlt er, wird auf `/login` weitergeleitet.

Reminder-Hub: `ReminderService` hält ein Signal mit den aktuell aktiven Erinnerungen und wird von `NavbarComponent` beim Öffnen der Glocke sowie bei jedem Login neu geladen. Das Dropdown schließt sich über eine `router.events`-Auswertung (`NavigationStart`) beim Reiterwechsel und über einen `HostListener('document:click')` bei Klicks außerhalb.

Client-seitige Validierung: `frontend/src/app/core/validation.ts` spiegelt bewusst dieselben Regeln wie `backend/app/validation.py`, damit Nutzer eine Fehlermeldung direkt am betroffenen Feld sehen, statt erst nach einem Server-Roundtrip. Der Server bleibt die verbindliche Instanz.

5. Deployment-Infrastruktur

■ 5.1 Lokale Entwicklungsumgebung

```
Docker Compose → PostgreSQL 16 (Port 5432)
Python 3.12 + virtualenv → Flask Dev-Server (Port 5000)
Node.js 22 + npm → Angular Dev-Server (Port 4200, Proxy auf 5000)
```

■ 5.2 Produktionsumgebung (Railway)

Railway ist eine Plattform-as-a-Service, die Docker-Container in der Cloud betreibt und automatisch bei jedem Push auf `main` neu baut (GitHub-Integration, kein `railway up` nötig).

Build-Prozess (Dockerfile):

1. Railway liest `Dockerfile` aus dem Repository-Root (gepinnt über `railway.json`, `"builder": "DOCKERFILE"`) — bis 2026-08-20 baute das Projekt stattdessen über Nixpacks; dieser Ansatz wurde verworfen, weil Railway den Nixpacks-BUILDER inzwischen als veraltet einstuft (siehe `docs/ExecPlans/completed/`, Plan P13).
2. Erste Build-Stufe (`node:22-slim`): `npm ci` installiert die Angular-Abhängigkeiten, `npm run build` erstellt den Angular-Produktions-Build (Output: `frontend/dist/frontend/browser/`).
3. Zweite Build-Stufe (`python:3.12-slim`): `pip install -r backend/requirements.txt` installiert die Backend-Abhängigkeiten; der Backend-Code und das aus der ersten Stufe gebaute Frontend werden in das Image kopiert.

Start-Prozess (`start.sh`):

1. `flask db upgrade` spielt ausstehende Datenbankmigrationen ein (idempotent).
2. `gunicorn wsgi:application -w 2 -b 0.0.0.0:$PORT` startet den Produktionsserver.

Statische Dateien: In Produktion liefert Flask die Angular-Dateien aus `frontend/dist/frontend/browser/` aus (`_register_spa_fallback` in `backend/app/__init__.py`). Alle Anfragen, die nicht mit `/api/` beginnen, werden auf `index.html` weitergeleitet (SPA-Routing) — es gibt deshalb nur einen Railway-Dienst für Frontend und Backend zusammen.

Umgebungsvariablen: Alle Secrets (Datenbankpasswort, JWT-/Flask-Secret) werden als Railway Environment Variables konfiguriert — nie im Repository gespeichert. Fehlt `SECRET_KEY` in Produktion, verweigert die Anwendung beim Start explizit den Dienst (`backend/app/__init__.py`), statt still auf einen im Quelltext stehenden Vorgabewert zurückzufallen.

■ 5.3 CI/CD-Pipeline (GitHub Actions)

Bei jedem Push auf `main`, `feature/**`, `fix/**`, `docs/**`, `FR-/**` sowie bei jedem Pull Request gegen `main` (`.github/workflows/ci.yml`):

1. Backend-Job: `ruff check` (Linting) + `pytest` (138 Tests, Stand 2026-08-21)

2. Frontend-Job: `ng lint` (ESLint) + `ng test --watch=false` (Vitest, 43 Tests, Stand 2026-08-21)

Der Deploy-Schritt hängt an beiden Prüf-Jobs (`needs: [backend, frontend]`) und läuft nur bei grüner Pipeline. Playwright-E2E-Tests (`frontend/e2e/`) sind **nicht** Teil der CI-Pipeline; sie werden manuell gegen eine laufende Umgebung ausgeführt (siehe Testabschlussbericht).